
Oliver Krauss

Modern Code - Introduction

ModernCode | MTD.ba VZ SS25

Hagenberg 06.10.2025

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

What is Programming?

- Giving a computer precise, step-by-step instructions
- Everywhere: ATMs, phones, games, cars, smart homes
- Typical flow: **Input** → **Processing** → **Output**
- Why learn it?
 - Problem solving, automation, creativity

Programming: Specs $\rightarrow$ Algorithms $\rightarrow$ Programs

- **Specification** (what to do): clear requirements and constraints
- **Algorithm** (how to do it): step-by-step, language-agnostic recipe
- **Program** (executable form): algorithm expressed in a programming language
- Cycle: Spec $\rightarrow$ Algorithm $\rightarrow$ Program $\rightarrow$ Test $\rightarrow$ Improve
- Example (game): Spec says *"If enemy health ≤ 0 , display 'You Win'"*; algorithm checks a condition; program uses **if** and **println**

Specification (What should happen?)

- **Definition:** Precise description of desired behavior, inputs/outputs, and constraints
- **Audience:** Humans (stakeholders, developers, testers)
- **Form:** User stories, acceptance criteria, pre/postconditions, examples
- **Example:** "When the goblin's HP reaches 0, display 'You Win'."

Algorithm (How to do it?)

- **Definition:** Language-agnostic, step-by-step procedure to satisfy the spec
- **Forms:** Pseudocode, flowcharts, decision tables
- **Qualities:** Correctness, termination, clear handling of all branches and edge cases
- **Example steps:**
 - Check enemy HP;
 - if $HP \leq 0$ then
 - show "You Win";
 - else
 - continue battle

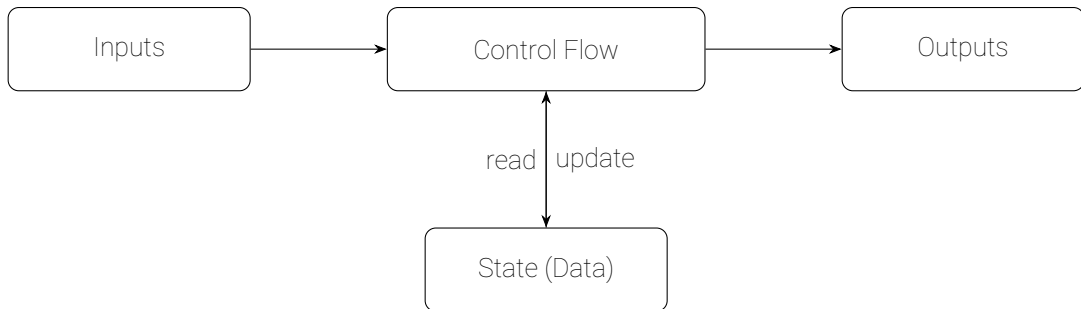
Program (Executable code)

- **Definition:** The algorithm expressed in a programming language (e.g., Java)
- **Artifacts:** Source code, unit tests, logging, error handling
- **Concerns:** Readability, maintainability, performance, correctness
- **Traceability:** Code should map back to steps in the algorithm and acceptance criteria in the spec

Example (Java):

```
1  int enemyHealth = 10; // example value
2
3  if (enemyHealth <= 0) {
4      IO.println("You Win");
5  } else {
6      IO.println("Fight on!");
7  }
```

Programs



Inputs, State, and Control

- **Inputs:** keyboard, files, network, random numbers
- **Outputs:** screen text, files, network responses, sounds, visuals
- **State (Data):** variables holding current values (e.g., **playerHealth**)
- **Control:** sequence (*do steps in order*), selection (*if/else*), iteration (*loops*)
- Programs transform inputs and current state into outputs by following control flow

Inputs

- **What are inputs?** Data that enters the program from the outside world
- **Sources:**
 - Keyboard / mouse
 - Files
 - Network
 - Sensors
 - App / form inputs
- **Example:** Read user actions from the keyboard and mouse

Outputs

- **What are outputs?** Results a program produces for users or other systems
- **Forms:**
 - Screen text (console)
 - Windows / buttons (GUI)
 - Files
 - Responses over the internet (API)
 - Images / sounds
- **Example:** print "You Win" on the screen

State (Data)

- **What is state?** The current values your program stores and updates over time
- **Memory:**
 - Stack: where short-lived values live while code runs
 - Heap: where objects live (we will learn more later)
- **Example:** `int enemyHealth, Player player, List<Enemy> enemies`

Control Flow

- **Sequence**: do steps in order
- **Selection**: `if/else`, `switch` to choose a branch
- **Iteration**: `for`, `while`, `do-while` to repeat
- **Exceptions**: alter normal flow on errors; use sparingly and handle close to source
- **Example**: decide win/lose based on `enemyHealth`

Qualities of Good Programs

- **Correctness**: does what it should be doing
- **Readability**: clear names, consistent formatting
- **Maintainability**: easy to modify and extend
- **Testability**: behavior can be verified
- **Performance**: fast enough
- **Robustness**: handles bad input gracefully

What is a Programming Language?

- A language for communicating with computers
- Defines syntax and semantics
- Provides tools for building programs
- **Examples:** Java, C++, Python, JavaScript, C#, Ruby
- Each has strengths: Java (enterprise), Python (data science), C++ (games)

Syntax vs Semantics I

Syntax = Grammar rules (how to write it correctly)

Semantics = Meaning (what it does when it runs)

Java:

```
1  if (enemyHealth <= 0) {  
2      IO.println("You Win");  
3  }
```

Syntax vs Semantics II

Python:

```
1  if enemy_health <= 0:  
2      print("You Win")
```

C++:

```
1  if (enemyHealth <= 0) {  
2      cout << "You Win" << endl;  
3  }
```

Same meaning, different syntax!

Java: Strengths and Weaknesses

Strengths:

- **Platform Independence:** "Write once, run anywhere"
- **Strong Typing:** Catches errors at compile time
- **Memory Management:** Automatic garbage collection
- **Rich Ecosystem:** Extensive libraries and frameworks
- **Object-Oriented:** we continue using it next semester

Weaknesses:

- **Verbose:** More code required for simple tasks
- **Performance Overhead:** JVM adds some overhead
- **Memory Usage:** Higher memory consumption than C/C++
- **Startup Time:** Slower initial execution

Python: Strengths and Weaknesses

Strengths:

- **Readable Syntax:** Code reads like English
- **Rapid Development:** Quick to write and test
- **Versatile:** Web, data science, AI, automation
- **Rich Libraries:** NumPy, Pandas, Django, Flask
- **Cross-Platform:** Runs on any operating system

Weaknesses:

- **Performance:** Slower than compiled languages
- **Memory Usage:** Can be memory-intensive
- **Terrible for learning:** no typing, no errors, no discipline
- **whitespace sensitive:** no indentation, no newlines, no spaces

C++: Strengths and Weaknesses

Strengths:

- **High Performance:** Direct hardware access, no overhead
- **Memory Control:** Manual memory management - can perform well
- **System Programming:** Operating systems, drivers, games
- **Compiled:** Fast execution, optimized code
- **Multi-Paradigm:** Procedural, OOP, generic programming

Weaknesses:

- **Complex Syntax:** Steep learning curve
- **Memory Management:** Manual allocation/deallocation
- **Platform Dependent:** Must recompile for each platform
- **Error-Prone:** Easy to introduce bugs and crashes

Programming Paradigms (Context)

- **Procedural** (our focus): step-by-step functions and data
- Object-Oriented (Next semester): classes, objects, state
- Functional: expressions, immutability
- ...
- Why procedural first? Simpler mental model; maps well to algorithms

Procedural Example (step-by-step)

```
1  int enemyHealth = 10;  
2  
3  if (enemyHealth <= 0) {  
4      IO.println("You Win");  
5  } else {  
6      IO.println("Fight on!");  
7  }
```

Sequence + selection: do steps in order and choose a branch.

Object-Oriented Example (objects with state)

```
1  class Enemy {  
2      private int health;  
3      Enemy(int health) { this.health = health; }  
4      boolean isDefeated() { return health <= 0; }  
5  }  
6  
7  Enemy goblin = new Enemy(10);  
8  if (goblin.isDefeated()) {  
9      IO.println("You Win");  
10 }
```

Objects bundle data (health) with behavior (isDefeated).

Functional Example (compute without changing data)

```
1  // Using streams for a sum (idea: functions, avoid changing state)
2  int[] numbers = {1, 2, 3, 4};
3  int sum = java.util.Arrays.stream(numbers)
4                                .map(x -> x * 2)
5                                .sum();
6  IO.println(sum);
```

Emphasis on expressions and transformations, not mutating shared state.

Overview of Java

- Mature, widely used language; platform independent via the JVM
- Strong typing, rich standard library, huge ecosystem
- Where it runs: backend services, Android, desktop tools, embedded
- This course: Java SE, procedural subset

Why Java?

- **Clear syntax:** easy to read and understand
- **Strong typing:** catches errors early, teaches good habits
- **Excellent tools:** IntelliJ IDEA makes learning easier
- **Real-world relevance:** used in industry and other courses
- **Procedural foundation:** perfect for learning basic programming concepts

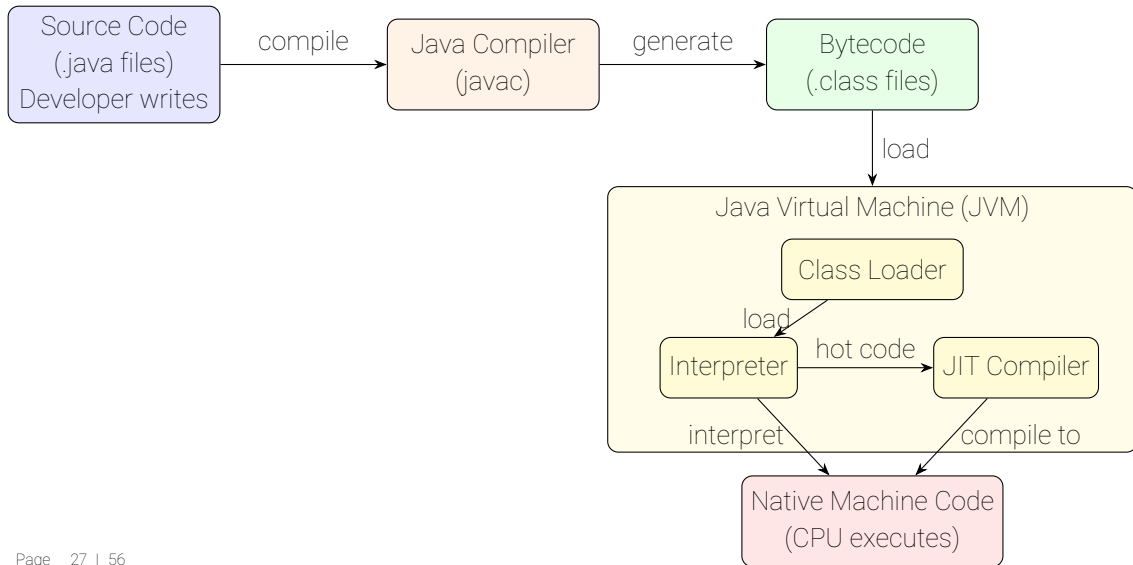
Java Features We'll Use

- **Basic types:** `int`, `double`, `boolean`, `String`
- **Variables:** storing and updating data
- **Control structures:** `if/else`, loops, methods
- **Input/Output:** reading from keyboard, writing to screen
- **Arrays:** storing collections of data
- **File handling:** focus on console programs first
- **Graphics/GUI:** cminimal GUI applications so you see what you code

What We Won't Cover (Yet)

- **Classes and Objects**: that's for next semester
- **Advanced libraries**: we'll stick to basics
- **Networking**: local programs only

From Source to Running Program



Key Terms

Compiler (javac) Translates source code (**.java**) to bytecode (**.class**)

Bytecode Platform-neutral instructions stored in **.class** files

JVM Java Virtual Machine that loads and executes bytecode

Class Loader JVM component that loads classes into memory

Interpreter JVM component that executes bytecode step-by-step

JIT Compiler Just-In-Time compiler that converts hot code to native machine code

Native Code Machine-specific instructions that the CPU executes directly

Hello, World! (Structure)

```
1  public class HelloWorld {  
2      public static void main(String[] args) {  
3          IO.println("Hello, World!");  
4      }  
5  }  
6
```

Structure Explanation:

- **Class:** Container for program code (name must match filename)
- **Main Method:** Entry point where execution starts
- **IO.println:** Prints text to console and moves to new line

Development Tools We Will Use

- **IntelliJ IDEA**: primary IDE for editing and running code
- **Copilot for Students (IntelliJ)**: AI integration
- **ChatGPT**: For explaining source code
- **Terminal (preview)**: `javac HelloWorld.java` then `java HelloWorld`

AI in Software Engineering (Short Overview)

- Where AI helps: clarifying requirements, drafting code, docs, tests, debugging, refactoring
- **Human-in-the-loop**: you design, verify, and own the results
- Limits: possible mistakes (*hallucinations*), missing context, licensing considerations
- We will learn to use AI responsibly and effectively!
 - Using AI is a skill that will be useful for the rest of your life
 - Using AI for your homework is ALLOWED!
 - Try to use AI to help you understand the code, not to write the code for you (yet)
 - AI is a tool, not here to replace us

Levels of AI Use in Software Engineering

Vibe Coding

Intuitive, exploratory coding supported by AI feedback and quick iterations.

Vibe Engineering

Aligning tools, workflow, and goals so the *experience* of building is smooth and effective.

Context Engineering

Curating prompts, examples, tests, and environment (IDE settings, docs) to get better outcomes from humans & AI.

We'll revisit these ideas later in the course.

What is Pseudocode?

- **Definition:** A step-by-step description of an algorithm in plain English
- **Purpose:** Focus on logic and problem-solving, ignore programming syntax
- **Benefits:**
 - Easier to read and understand than code
 - Language-independent (works for Java, Python, C++, etc.)
 - Helps catch design flaws before writing actual code
 - Great for communicating with non-programmers

Why Use Pseudocode?

- **Planning:** Think through your solution before coding
- **Communication:** Explain algorithms to teammates or stakeholders
- **Debugging:** Find logical errors before syntax errors
- **Documentation:** Create clear descriptions of how your code works
- **Learning:** Break complex problems into manageable steps

Pseudocode Writing Rules

- **Capitalize keywords:** IF, THEN, ELSE, WHILE, FOR, etc.
- **One statement per line:** Keep it simple and readable
- **Indent properly:** Show hierarchy and nested structures
- **Use END keywords:** ENDIF, ENDWHILE, ENDFOR to close blocks
- **Keep it simple:** Use clear, descriptive names
- **Be consistent:** Use the same style throughout

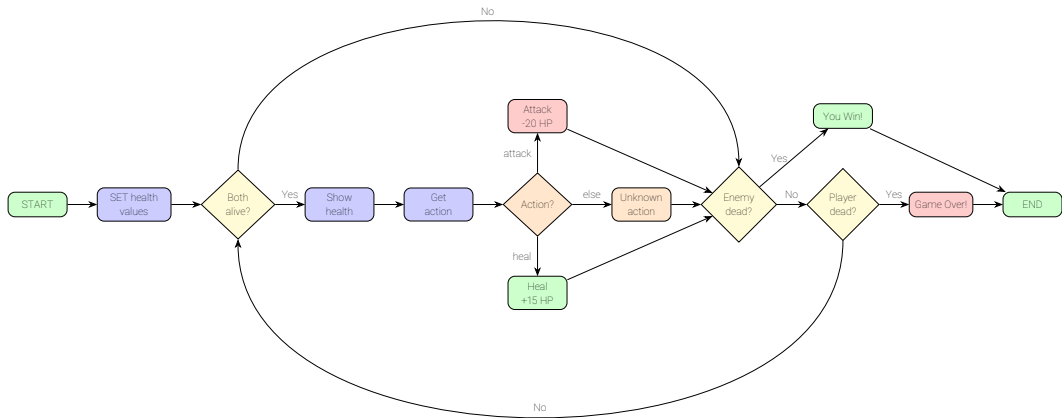
Common Pseudocode Constructs

- **SEQUENCE**: Do steps in order
- **IF-THEN-ELSE**: Make decisions based on conditions
- **WHILE**: Repeat while a condition is true
- **FOR**: Repeat a specific number of times
- **REPEAT-UNTIL**: Repeat until a condition becomes true
- **CASE**: Choose from multiple options

The Task: Simple Battle Game

- **Goal:** Create a turn-based battle game between player and enemy
- **Starting Conditions:** Player has 100 health, enemy has 50 health
- **Player Actions:**
 - **Attack:** Deal 20 damage to enemy
 - **Heal:** Restore 15 health to player
- **Game Rules:**
 - Battle continues until someone's health reaches 0
 - Player chooses action each turn
 - Display current health status each turn
 - Announce winner when battle ends

Simple Battle Game Flowchart



Pseudocode: Setup and Main Loop

```
1  START
2  SET playerHealth TO 100
3  SET enemyHealth TO 50
4
5  WHILE playerHealth > 0 AND enemyHealth > 0
6      ...
7  ENDWHILE
8  END
```

-- Initialize starting values

-- Create main game loop condition

Pseudocode: Display and Input

```
1  WHILE playerHealth > 0 AND enemyHealth > 0
2      OUTPUT "Your health: " + playerHealth
3      OUTPUT "Enemy health: " + enemyHealth
4      READ playerAction
5      ...
6  ENDWHILE
```

-- Show current game state

-- Get player's action choice

Pseudocode: Action Processing

```
1  IF playerAction == "attack" THEN
2      SET damage TO 20
3      SET enemyHealth TO enemyHealth - damage
4      OUTPUT "You attack! Enemy takes " + damage + " damage"
5  ELSE IF playerAction == "heal" THEN
6      SET healAmount TO 15
7      SET playerHealth TO playerHealth + healAmount
8      OUTPUT "You heal for " + healAmount + " health"
9  ELSE
10     OUTPUT "Unknown action, try again"
11  ENDIF
```

-- Handle attack, heal and invalid input

Pseudocode: Win/Lose Conditions

```
1  IF enemyHealth <= 0 THEN
2      OUTPUT "You win! Enemy defeated!"
3      BREAK
4  ENDIF
5
6  IF playerHealth <= 0 THEN
7      OUTPUT "Game Over! You were defeated!"
8      BREAK
9  ENDIF
```

- Check if enemy is defeated
- Check if player is defeated
- End game when conditions are met

Example Pseudocode: Simple Battle (Full)

```
1  START
2  SET playerHealth TO 100
3  SET enemyHealth TO 50
4  WHILE playerHealth > 0 AND enemyHealth > 0
5      OUTPUT "Your health: " + playerHealth
6      OUTPUT "Enemy health: " + enemyHealth
7      READ playerAction
8      IF playerAction == "attack" THEN
9          SET damage TO 20
10         SET enemyHealth TO enemyHealth - damage
11         OUTPUT "You attack! Enemy takes " + damage + " damage"
12     ELSE IF playerAction == "heal" THEN
13         SET healAmount TO 15
14         SET playerHealth TO playerHealth + healAmount
15         OUTPUT "You heal for " + healAmount + " health"
16     ELSE
17         OUTPUT "Unknown action, try again"
18     ENDIF
19     IF enemyHealth <= 0 THEN
20         OUTPUT "You win! Enemy defeated!"
21         BREAK
22     ENDIF
23     IF playerHealth <= 0 THEN
24         OUTPUT "Game Over! You were defeated!"
25         BREAK
26     ENDIF
27 ENDWHILE
28 END
```

Simple Battle in Java I

```
1  int playerHealth = 100;
2  int enemyHealth = 50;
3
4  while (playerHealth > 0 && enemyHealth > 0) {
5      IO.println("Your health: " + playerHealth);
6      IO.println("Enemy health: " + enemyHealth);
7      String playerAction = IO.readLine(); // simplified
```

Simple Battle in Java II

```
9      if (playerAction.equals("attack")) {
10          int damage = 20;
11          enemyHealth -= damage;
12          IO.println("Enemy takes " + damage + " damage");
13      } else if (playerAction.equals("heal")) {
14          int healAmount = 15;
15          playerHealth += healAmount;
16          IO.println("Heal for " + healAmount + " health");
17      } else {
18          IO.println("Unknown action, try again");
19      }
```

Simple Battle in Java III

```
20     if (enemyHealth <= 0) {
21         IO.println("You win! Enemy defeated!");
22         break;
23     }
24     if (playerHealth <= 0) {
25         IO.println("Game Over! You were defeated!");
26         break;
27     }
28 }
```

What is Abstraction?

- **Abstraction** = Hiding unnecessary details to focus on what matters
- Think of it like a **map**: you don't see every tree, just the roads and landmarks
- In programming: focus on *what* something does, not *how* it does it
- **Example**: "Make coffee" vs "Boil water, grind beans, pour water, wait 4 minutes..."

Why use abstraction?

- **Clarity**: Easier to understand the big picture
- **Planning**: Think about goals before implementation
- **Communication**: Explain ideas to others simply
- **Debugging**: Find problems at the right level

What is Decomposition?

- **Decomposition** = Breaking a big problem into smaller, manageable pieces
- Like **recipe steps**: "Make dinner" becomes "chop vegetables, cook meat, set table"
- Each piece should be:
 - **Understandable** on its own
 - **Testable** independently
 - **Reusable** in other contexts

Example: Battle Game

- RUN BATTLE GAME → INITIALIZE + LOOP + CHECK
- HANDLE TURN → READ INPUT + PROCESS ACTION + UPDATE STATE
- PROCESS ACTION → ATTACK or HEAL or ERROR

Abstraction Levels: Battle Game Example

Level 1: Very Abstract (What we want)

- RUN BATTLE GAME
- PLAYER FIGHTS ENEMY
- SOMEONE WINS

Level 2: Less Abstract (Major steps)

- INITIALIZE player and enemy
- LOOP until someone wins
- HANDLE TURN for player
- CHECK win/lose conditions

Level 3: Detailed (How to do it)

- SET playerHealth TO 100
- WHILE playerHealth > 0 AND enemyHealth > 0
- IF playerAction == "attack" THEN SET damage TO 20

Top-Down Approach (Start Abstract, Get Specific)

How it works:

1. Start with the **big goal** (very abstract)
2. Break it into **major components** (less abstract)
3. Break each component into **smaller steps** (more specific)
4. Keep breaking until you reach **implementable code**

Battle Game Example:

- RUN BATTLE GAME (Goal)
- ↓ Break into: INITIALIZE + LOOP + CHECK
- ↓ Break LOOP: STATUS + INPUT + ACTION + UPDATE
- ↓ Break ACTION: ATTACK or HEAL or ERROR
- ↓ Implement: IF `playerAction.equals("attack")` THEN...

When to use: Planning new programs, understanding requirements

Bottom-Up Approach (Start Specific, Build Up)

How it works:

1. Start with **small, working pieces** (very specific)
2. Combine pieces into **larger components** (less specific)
3. Combine components into **complete systems** (more abstract)
4. Test each level as you build up

Battle Game Example:

- Start with: `IO.println("Health: " + playerHealth)` (working output)
- Build: `displayStatus()` method (combines multiple outputs)
- Build: `handleTurn()` method (combines input + action + status)
- Build: `gameLoop()` method (combines all turn handling)
- Build: `main()` method (combines initialization + game loop)

When to use: Debugging existing code, building reusable components

Which Approach Should You Use?

Top-Down is better when:

- **Starting a new project** - you need a plan
- **Understanding problems** - what should the program do?
- **Breaking down complex problems** - how do the pieces fit?
- **Communicating with others** - explain the big picture first

Bottom-Up is better when:

- **Debugging existing code** - find what's broken
- **Building reusable components** - create tools for later
- **Testing individual parts** - make sure pieces work before combining
- **Learning new concepts** - understand the building blocks first

In practice: Most good programmers use **both approaches** together!

AI Demo: Explain HelloWorld

Explain each line of this Java program in one sentence. Explain the keywords and the purpose of each line.

```
1  public class HelloWorld {  
2      public static void main(String[] args) {  
3          IO.println("Hello, World!");  
4      }  
5  }
```

AI Demo: Algorithm Design

Help me design an algorithm for a simple battle game. Start with a very abstract description, then break it down step by step into smaller, manageable pieces. Use a top-down approach.

The task: Use AI to help you:

- Start with the big goal: "Create a battle game"
- Break it into major components
- Break each component into smaller steps
- Keep breaking until you have specific, implementable actions

Summary: Key Takeaways

- **Programming** = Combining input, state, output, and control flow to solve problems
- **Java**: A widely-used, strongly-typed, object-oriented language
- **Syntax vs. Semantics**: Syntax is how you write code; semantics is what it means
- **Programming Paradigms**: Procedural (focus now), Object-Oriented, Functional
- **Pseudocode**: Plan your logic before coding; focus on clarity and structure
- **AI Tools**: Use AI to help explain, debug, and design code—but always verify results
- **Top-Down vs. Bottom-Up**: Use both approaches for effective problem solving

Next Steps

- **Practice:** Install IntelliJ & Java, Sign up for GitHub Education and ChatGPT
- **Explore:** Experiment with AI tools for code explanation and design
- **Prepare:** Review today's examples and concepts for next class
- **Questions?** Bring them to the next session or post on the course forum!

Happy coding!